

Aletheia

Research intelligence workspace - implementation snapshot

1. Project Overview

Mission

Turn a research library into auditable answers.

Aletheia ingests project sources, extracts structured paper content, retrieves evidence semantically, reranks it, and returns grounded answers with traceable page and section citations.

Value proposition



What it is

- Research-paper library and reader
- Project-scoped semantic search
- Cited question answering and agent research

What it is not

- A generic chat-with-PDF demo
- An uncited answer generator
- A replacement for researcher judgement

2. Target Audience / Users

Researchers

Compare papers, inspect source passages, and synthesize evidence.

Students

Read technical literature with visible provenance and page-level context.

Research teams

Maintain project-specific libraries, questions, and documented findings.

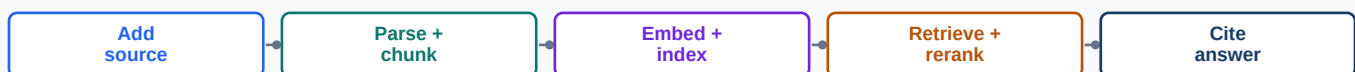
Technology map

Frontend, backend, retrieval, storage, and model operations

3. Tech Stack & Tools

Experience	Next.js 16 - TypeScript - Tailwind CSS
Application API	FastAPI - Pydantic - Python
Document intelligence	PyMuPDF - normalization - section-aware chunking
Retrieval	FastEmbed Jina embeddings - pgvector - local reranker
Generation	OpenRouter LLM for grounded answers, agent planning, figure interpretation
Persistence	Postgres 17 - local source storage - Chroma mirror where configured

Evidence flow



Design controls

Grounding: the backend issues evidence IDs and resolves citations; model-invented IDs are removed. **Resilience:** ingestion jobs expose stage, progress, failure, and retry state. **Security:** API keys remain server-side environment variables.

Features and current status

Implemented capabilities, preview surfaces, and operating constraints

4. Core Features

Project sources

Projects, source inventory, upload, delete, processing status

LIVE

Reading workspace

Library filters, extracted pages, section outline, figures/tables/assets

LIVE

Semantic retrieval

Embedded chunks, project scope, page/section-aware results

LIVE

Ask and citations

Retrieve -> rerank -> evidence context -> cited response

LIVE

Research Agent

Bounded subquestions, source discovery, grounded synthesis

LIVE - KEY REQUIRED

Claim / cross-paper / reproducibility

Verification and comparison workflows

PREVIEW / IN PROGRESS

5. Current Status

Snapshot

Core workspace: operational locally - source ingestion, library, reader, semantic retrieval, Ask, and the bounded Research Agent are wired.

Model operations: local embeddings/reranking and research diagnostics are present; hosted-generation routes require an OpenRouter key.

Product maturity: active implementation - core RAG is usable; verification and reproducibility surfaces need completion and live-data hardening.

Three Colab model-training notebooks

Purpose, architecture, validation design, and production role

Colab notebook portfolio

1. Neural Reranker

File: Aletheia_Neural_Reranker_Colab.ipynb

Architecture: 5 -> 16 -> 8 -> 1 PyTorch MLP

Purpose: Trains a compact query-passage relevance reranker over a pinned 10-paper arXiv corpus and 150 benchmark questions.

Evaluation: Question-ID held-out split; MRR and Recall@6 versus cosine baseline; SHA-256 corpus validation.

Production role: Candidate experiment - does not replace production cross-encoder.

2. Multi-source Relevance

File: Aletheia_Multisource_Relevance_Colab.ipynb

Architecture: 516 -> 96 -> 32 -> 2 NumPy network

Purpose: Learns whether a query and passage are relevant using SciQ and SQuAD public pairs.

Evaluation: 70/15/15 connected-group split; leakage checks; weighted loss, early stopping, held-out macro F1.

Production role: Integrated only as experimental answer diagnostics.

3. Scientific Stance

File: Aletheia_Scientific_Stance_Colab.ipynb

Architecture: 516 -> 128 -> 48 -> 2 NumPy network

Purpose: Classifies support vs contradiction on annotated SciFact abstract pairs.

Evaluation: Same split discipline; confusion matrix, class F1, validation-selected checkpoint.

Production role: Experimental only - no neutral class and not a fact checker.

Model signal and next priorities

Measured outcomes, limitations, and a practical sequence to completion

Training results - experimental diagnostics

Model	Test accuracy	Test macro F1	Baseline accuracy
Relevance classifier	89.87%	0.8987	84.96%
Scientific stance	54.00%	0.4715	63.00%

Interpretation guardrails

Relevance improves on its lexical baseline and is useful as a diagnostic signal. Stance accuracy trails its majority baseline; contradiction performance is weak. Neither model determines truth, replaces the Jina reranker, overrides LLM evidence, or supplies calibrated confidence.

6. Next Steps & Priorities

P0 - Stability

Keep local API, database migrations, and source deletion reliable; add regression coverage for source types and lifecycle.

P1 - Product completion

Replace preview claim, cross-paper, and reproducibility pages with live API-backed workflows.

P1 - Search quality

Benchmark source-scoped retrieval, chunk settings, reranking, and failure fallback against a fixed question set.

P2 - Research operations

Run the Colabs in hosted Colab, retain artifacts, and only promote models after end-to-end evaluation.

P2 - Deployment readiness

Add authentication, production storage, observability, secrets management, and CI/CD checks.

Recommended delivery flow

A concise operating roadmap for the next project cycle

From working prototype to research-ready platform

